



KARACHI INSTITUTE OF ECONOMICS AND TECHNOLOGY

Department of Computer Science

PENETRATION TESTING GUIDE

Secure Student Portal

How to Perform and Block All Security Attacks

Subject: Secure Software Development

Submitted By: Ibraheem Ahmed | Roll No: 15762

Date: May 2026

TABLE OF CONTENTS

Introduction	3
Attack 1: SQL Injection.....	4
What is SQL Injection?	4
How to Perform This Attack	4
Why is it Blocked?	4
What to Show in Viva	4
Attack 2: XSS (Cross-Site Scripting)	6
What is XSS?	6
How to Perform This Attack	6
Why is it Blocked?	6
What to Show in Viva	7
Attack 3: Brute Force Attack.....	8
What is a Brute Force Attack?	8
How to Perform This Attack	8
Why is it Blocked?	9
What to Show in Viva	9
Attack 4: CSRF (Cross-Site Request Forgery)	10
What is CSRF?.....	10
How to Perform This Attack	10
Why is it Blocked?	10
What to Show in Viva	11
Attack 5: Unauthorized Access	12
What is Unauthorized Access?	12
Test 5A — Access Without Login	12
Test 5B — Student Accessing Admin Panel	12
Test 5C — Admin Accessing Student Dashboard.....	12
Why is it Blocked?	13
What to Show in Viva	13
Attack 6: Privilege Escalation	14
What is Privilege Escalation?	14
How to Perform This Attack	14
Why is it Blocked?	14
What to Show in Viva	14
Quick Reference — All Attacks Summary	15
Log File Evidence	15

Introduction

This document is a step-by-step penetration testing guide for the Secure Student Portal project. It explains how each attack is performed and how the application blocks it. This guide is meant to be used during project demonstration and viva.

Before performing any test, make sure:

- The application is running: `py app.py`
- You are using Google Chrome or Firefox browser
- The app is accessible at: `http://127.0.0.1:5000`
- You have at least one admin and one student account registered

#	Attack	Tool Used	Expected Result
1	SQL Injection	Browser (Login Form)	Login blocked - payload treated as text
2	XSS Attack	Browser (Register Form)	Script not executed - shown as plain text
3	Brute Force	Browser (Login Form)	Account locked after 5 failed attempts
4	CSRF Attack	Browser Developer Console	Request blocked - missing CSRF token
5	Unauthorized Access	Browser (Direct URL)	Redirected to login page
6	Privilege Escalation	Browser (Login as Student)	Access denied to admin panel

Attack 1: SQL Injection

What is SQL Injection?

SQL Injection is an attack where a hacker enters special SQL code into a form field to manipulate the database query. For example, entering ' OR 1=1 -- in a username field can trick a database into logging in without a password because 1=1 is always true.

How to Perform This Attack

Step 1 — Open the login page

Go to: `http://127.0.0.1:5000/login`

Step 2 — Enter the SQL injection payload

In the Username field, type exactly this:

```
' OR 1=1 --
```

In the Password field, type anything:

```
anypassword
```

Step 3 — Click Login

Step 4 — Observe the result

BLOCKED: Application shows 'Invalid username or password'. You are NOT logged in.

Why is it Blocked?

Our application uses SQLAlchemy ORM which automatically uses prepared statements. This means the username input is treated as plain text data, not as SQL code. The query becomes:

```
SELECT * FROM users WHERE username = '' OR 1=1 --  
  
# The entire payload is treated as a string  
# No user exists with that username, so login fails
```

What to Show in Viva

- Type the payload in the username field
- Click Login
- Show that login was denied
- Open portal.log and show the failed login entry

```
WARNING - Failed login attempt for unknown user: ' OR 1=1 --
```


Attack 2: XSS (Cross-Site Scripting)

What is XSS?

XSS is an attack where a hacker injects malicious JavaScript into a web page. If successful, the script runs in the victim's browser and can steal cookies, redirect users, or show fake content.

How to Perform This Attack

Step 1 — Open the register page

Go to: `http://127.0.0.1:5000/register`

Step 2 — Enter the XSS payload as username

In the Username field, type exactly this:

```
<script>alert('XSS Attack!')</script>
```

Enter any valid password (min 6 characters):

```
test123
```

Step 3 — Click Register

Step 4 — Try to login with that username

Go to login page and enter the script tag as username and test123 as password.

Step 5 — Observe the result

BLOCKED: No popup appears. The script tag is shown as plain text, NOT executed as code.

Why is it Blocked?

Jinja2 templating engine (used by Flask) automatically escapes all output. This means `<` becomes `<`; and `>` becomes `>`; so the browser renders them as text characters, not HTML tags.

```
# What attacker entered:
<script>alert('XSS!')</script>

# What Jinja2 renders in the HTML page:
&lt;script&gt;alert('XSS!')&lt;/script&gt;

# Browser displays this as plain text:
<script>alert('XSS!')</script>

# The script is NEVER executed
```

What to Show in Viva

- Type the script tag in the register form username field
- Register the account
- Login with that username
- Show that you see the username displayed as text on the dashboard
- Show that no JavaScript popup appeared
- Right-click the page and click 'View Page Source' to show the escaped HTML

Attack 3: Brute Force Attack

What is a Brute Force Attack?

A brute force attack is when a hacker tries thousands of different passwords to guess the correct one. Without protection, an attacker can keep trying passwords until they get in.

How to Perform This Attack

Step 1 — Make sure you have a student account

Register an account with:

```
Username: teststudent  
Password: correct123
```

Step 2 — Go to the login page

Go to: <http://127.0.0.1:5000/login>

Step 3 — Enter wrong password repeatedly

Username: teststudent

Enter these wrong passwords one by one and click Login each time:

Attempt	Username	Password Entered	Expected Message
1st	teststudent	wrongpass1	Invalid password. 4 attempts remaining before logout.
2nd	teststudent	wrongpass2	Invalid password. 3 attempts remaining before logout.
3rd	teststudent	wrongpass3	Invalid password. 2 attempts remaining before logout.
4th	teststudent	wrongpass4	Invalid password. 1 attempts remaining before logout.
5th	teststudent	wrongpass5	Account locked after 5 failed attempts. Contact admin.

Step 4 — Try correct password after logout

Now try to login with the CORRECT password: correct123

BLOCKED: Even the correct password is rejected. Account is locked.

Step 5 — Unlock from Admin Panel

Login as admin and go to the admin dashboard. You will see the student account showing:

- Status: Locked (red badge)
- Failed Attempts: 5/5
- An Unlock button next to the account

Click Unlock to restore access.

Why is it Blocked?

The application tracks `failed_attempts` and `is_locked` fields in the database for each user. After 5 failures, the account is locked and cannot be accessed until an admin unlocks it.

```
# In database after 5 failed attempts:
failed_attempts = 5
is_locked = True

# Login check:
if user and user.is_locked:
    flash('Account locked. Contact admin.')
    return redirect(url_for('login')) # Blocked!
```

What to Show in Viva

- Perform all 5 failed login attempts live
- Show the countdown messages appearing each time
- Try the correct password after lockout - show it is still blocked
- Login as admin - show the locked badge and failed attempts counter
- Click unlock and show the student can login again
- Open `portal.log` and show the WARNING entries for each failed attempt

Attack 4: CSRF (Cross-Site Request Forgery)

What is CSRF?

CSRF is an attack where a malicious website sends requests to your web application on behalf of a logged-in user without their knowledge. For example, a hacker could create a fake website that secretly submits a form to your portal.

How to Perform This Attack

Step 1 — Open the login page

Go to: `http://127.0.0.1:5000/login`

Step 2 — Open Browser Developer Tools

Press F12 on your keyboard. The developer tools panel will open at the bottom or side of your browser.

Step 3 — Go to the Console tab

Click on the 'Console' tab in the developer tools panel.

Step 4 — Allow pasting (Chrome only)

Chrome shows a security warning. Type this and press Enter:

```
allow pasting
```

Step 5 — Paste the CSRF attack code

Copy and paste this code into the console and press Enter:

```
fetch('/login', {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'application/x-www-form-urlencoded'  
  },  
  body: 'username=admin1&password=admin123'  
}).then(r => r.text()).then(t => console.log(t))
```

Step 6 — Observe the result

Look at the console output. You will see the HTML of the page returned. Look for this message inside it:

```
Security error: Invalid or missing CSRF token. Request blocked.
```

```
BLOCKED: The request was rejected because it had no CSRF token. Login did not happen.
```

Why is it Blocked?

Flask-WTF adds a hidden CSRF token to every form. When a POST request comes in, it checks if the token matches. The JavaScript fetch request above did not include the token, so it was rejected.

```
# In the HTML form (legitimate request):
<input type='hidden' name='csrf_token' value='abc123xyz'>

# The fetch attack sends NO csrf_token
# Flask-WTF detects this and blocks the request

# Error handler fires:
@app.errorhandler(CSRFError)
def handle_csrf_error(e):
    flash('Security error: Invalid CSRF token. Blocked.')
    return redirect(url_for('login'))
```

What to Show in Viva

- Open the login page
- Open developer console (F12)
- Paste the fetch code and run it
- Show the CSRF error message in the console output
- Show that you are NOT logged in after the attack
- Then show the CSRF token in the page source (right click > View Page Source, search for csrf_token)

Attack 5: Unauthorized Access

What is Unauthorized Access?

An attacker tries to access protected pages directly via URL without logging in, or tries to access pages that belong to a different role.

Test 5A — Access Without Login

How to Perform

1. Make sure you are logged out
2. Directly type these URLs in the browser address bar:

```
http://127.0.0.1:5000/dashboard  
http://127.0.0.1:5000/admin  
http://127.0.0.1:5000/admin/student/1
```

Expected Result

BLOCKED: All URLs redirect to the login page automatically. Protected content is never shown.

Test 5B — Student Accessing Admin Panel

How to Perform

3. Login as a student account
4. After landing on the student dashboard, manually type in the address bar:

```
http://127.0.0.1:5000/admin
```

Expected Result

BLOCKED: Access Denied message is shown. Student is redirected away from admin panel.

Test 5C — Admin Accessing Student Dashboard

How to Perform

5. Login as an admin account
6. After landing on the admin panel, manually type in the address bar:

```
http://127.0.0.1:5000/dashboard
```

Expected Result

BLOCKED: Admin is redirected back to the admin panel. Student dashboard is not accessible.

Why is it Blocked?

Every route has two layers of protection:

- `@login_required` decorator — checks if user is logged in at all
- Role check inside the route — checks if the logged in user has the correct role

```
@app.route('/admin')
@login_required          # Layer 1: Must be logged in
def admin_dashboard():
    if current_user.role != 'admin': # Layer 2: Must be admin
        flash('Access denied.', 'danger')
    return redirect(url_for('student_dashboard'))
```

What to Show in Viva

- Logout completely
- Try to visit `/dashboard` and `/admin` directly - show redirect to login
- Login as student, then try `/admin` - show access denied
- Login as admin, then try `/dashboard` - show redirect to admin panel

Attack 6: Privilege Escalation

What is Privilege Escalation?

Privilege escalation is when a lower-privilege user (student) tries to gain higher-privilege access (admin) to perform actions they are not allowed to do, such as adding or deleting marks.

How to Perform This Attack

Step 1 — Login as a student

Login with a student account and land on the student dashboard.

Step 2 — Try to access admin-only routes directly

Type these URLs directly in the browser address bar:

```
http://127.0.0.1:5000/admin
http://127.0.0.1:5000/admin/student/1
```

Step 3 — Try to add marks via direct POST

Open developer console (F12) and try to submit a mark directly:

```
fetch('/admin/add_mark/1', {
  method: 'POST',
  headers: {'Content-Type': 'application/x-www-form-urlencoded'},
  body: 'subject=Hacked&marks=100&csrf_token=fake'
}).then(r => r.text()).then(t => console.log(t))
```

Expected Results

BLOCKED: All attempts are denied. Student cannot access any admin functionality.

Why is it Blocked?

- RBAC checks role on every single admin route
- CSRF token validation blocks the fake POST request
- Even if CSRF was bypassed, the role check would still deny access

What to Show in Viva

- Login as student
- Try to visit /admin - show access denied
- Run the fetch attack in console - show CSRF block
- Show that student's marks were NOT changed

Quick Reference — All Attacks Summary

Attack	Where to Test	Payload / Method	Result
SQL Injection	Login page - username field	' OR 1=1 --	BLOCKED - Invalid credentials
XSS	Register page - username field	<script>alert(1)</script>	BLOCKED - Shown as plain text
Brute Force	Login page - wrong password 5x	Any wrong password x5	BLOCKED - Account locked
CSRF	Browser console (F12)	fetch() POST without token	BLOCKED - CSRF token missing
Unauth Access	Browser address bar (logged out)	Direct URL to /dashboard or /admin	BLOCKED - Redirect to login
Privilege Escalation	Browser (logged in as student)	Direct URL to /admin	BLOCKED - Access denied

Log File Evidence

After performing all attacks, open portal.log in your project folder. You will see entries like:

```
2026-05-23 10:00:01 - WARNING - Failed login attempt for unknown user: ' OR 1=1 --
2026-05-23 10:01:05 - INFO      - New user registered:
<script>alert(1)</script> as student
2026-05-23 10:02:10 - WARNING - Failed login attempt 1/5 for: teststudent
2026-05-23 10:02:15 - WARNING - Failed login attempt 2/5 for: teststudent
2026-05-23 10:02:20 - WARNING - Failed login attempt 3/5 for: teststudent
2026-05-23 10:02:25 - WARNING - Failed login attempt 4/5 for: teststudent
2026-05-23 10:02:30 - WARNING - Failed login attempt 5/5 for: teststudent
2026-05-23 10:02:30 - WARNING - Account LOCKED: teststudent
2026-05-23 10:03:00 - WARNING - CSRF attack blocked: The CSRF token is missing.
```

This log file is your evidence that all attacks were detected and blocked. Show it during your viva.